



# I am starting on a Lab Practice and need your help completing it.

Some rules to abide by for this practice:

`case_match()` is not permitted in this lab because it is deprecated. Use `recode_values()` or `replace_values()` instead. You will likely need to consult the help pages for these functions, which serve as an opportunity to practice reading technical documentation.

Constants and special values `TRUE`, `FALSE`, `NA`, `NULL`

Assignment, comments, packages `<-` (assignment), `=` (named arguments), `#` (comment), `library()`, `~` (tilde for formulas, required for `recode_values()` and `replace_values()`)

Constructing data `c()`, `tibble()`, `seq()`, `:` (sequence)

Inspecting structure `class()`, `nrow()`, `ncol()`, `length()`, `glimpse()`, `head()`, `tail()`, `unique()`

Type predicates `is.numeric()`, `is.character()`, `is.logical()`, `is.factor()`, `is.ordered()`, `is.data.frame()`, `is.na()`

Arithmetic operators `+`, `-`, `*`, `/`, `^`, `**`, `%/%`, `%%`

Comparison and logical operators `==`, `!=`, `<`, `>`, `<=`, `>=`, `&`, `|`, `!`, `%in%`

Subsetting and extraction `[]`, `$`, `pull()`

Aggregation and summary `sum()`, `mean()`, `min()`, `max()`, `all()`, `any()`

Identity and ranking `identical()`, `min_rank()`, `desc()`

Conditional value selection `if_else()`, `recode_values()`, `replace_values()`

Factor handling `factor()`, `levels()`, `fct_recode()`, `fct_collapse()`, `fct_other()`, `fct_infreq()`, `fct_count()`

Data import `read_csv()`, `read_xls()`, `read_xlsx()`, `read_excel()`

Pipe and namespace `|>` (pipe), `::` (namespace qualifier)

Enforce this at every juncture of project

Understood! I've noted all the rules and constraints for your Lab Practice. Here's a clear summary of the guidelines I'll enforce throughout our work together:

## Lab Rules & Permitted Functions

Prohibited:

- `case_match()` — deprecated; use `recode_values()` or `replace_values()` instead [\[1\]](#) [\[2\]](#)

Permitted Functions by Category:

| Category                    | Permitted Functions/Operators                                                                   |
|-----------------------------|-------------------------------------------------------------------------------------------------|
| Constants & Special Values  | TRUE, FALSE, NA, NULL                                                                           |
| Assignment & Comments       | <-, =(named args), #, library(), ~(tilde/formulas)                                              |
| Constructing Data           | c(), tibble(), seq(), :                                                                         |
| Inspecting Structure        | class(), nrow(), ncol(), length(), glimpse(), head(), tail(), unique()                          |
| Type Predicates             | is.numeric(), is.character(), is.logical(), is.factor(), is.ordered(), is.data.frame(), is.na() |
| Arithmetic                  | +, -, *, /, ^, **, %/%, %%                                                                      |
| Comparison & Logical        | ==, !=, <, >, <=, >=, &, \  , !, %in%                                                           |
| Subsetting & Extraction     | [ ], \$, pull()                                                                                 |
| Aggregation & Summary       | sum(), mean(), min(), max(), all(), any()                                                       |
| Identity & Ranking          | identical(), min_rank(), desc()                                                                 |
| Conditional Value Selection | if_else(), recode_values(), replace_values()                                                    |
| Factor Handling             | factor(), levels(), fct_recode(), fct_collapse(), fct_other(), fct_infreq(), fct_count()        |
| Data Import                 | read_csv(), read_xls(), read_xlsx(), read_excel()                                               |
| Pipe & Namespace            | \  > (pipe), :: (namespace qualifier)                                                           |

## Key Notes on `recode_values()` VS `replace_values()`

- Use `recode_values()` when creating an **entirely new vector** — unmatched values fall through to a default<sup>[2]</sup>
- Use `replace_values()` when **partially updating** an existing vector — unmatched values retain their original values<sup>[2]</sup>
- Both support formula syntax: `recode_values(x, old1 ~ new1, old2 ~ new2)`<sup>[2]</sup>
- Both also support a vector-based lookup table: `recode_values(x, from = from_vec, to = to_vec)`<sup>[2]</sup>

I'm ready to help you complete the lab! Please go ahead and share the lab questions or tasks, and I'll make sure every solution strictly follows these rules.

✱✱

# This is the lab practice which I need completed. Do focus on the object of the lab and give guide to complete it

I've read the full lab guide. Here is a comprehensive guide to completing every task in Section 3 (ASEAN Meat Production 2018–2021), strictly using only the permitted toolbox.

## Setup

Place this at the top of your QMD file:

```
#| label: setup-meat
#| message: false
library(tidyverse)
meat <- read_csv("meat.csv")
```

## Section 3.1 — Inspect the `meat` Tibble

Task 1 (predictions first, then verify):

```
#| label: inspect-classes
# Column classes
class(meat$country)
class(meat$animal)
class(meat$year)
class(meat$production)
```

**Expected:** `country` and `animal` → "character", `year` → "numeric", `production` → "numeric".

```
#| label: inspect-shape
# Dimensions
nrow(meat)
ncol(meat)
```

```
#| label: inspect-unique
# Distinct value counts
length(unique(meat$country)) # Expect 10
length(unique(meat$animal))  # Expect 4
length(unique(meat$year))    # Expect 4 (2018–2021)
```

**Verifier cross-check**— use `glimpse()` as a second route:

```
#| label: inspect-glimpse
glimpse(meat)
```

### Task 3 — head and tail:

```
#| label: inspect-headtail
head(meat)
tail(meat)
```

Note observations in the QMD about ordering (e.g., alphabetical by country, then animal, then year).

## Section 3.2 — Handle Missing Values

### Task 1 — confirm NAs exist:

```
#| label: missing-confirm
any(is.na(meat$production))
```

### Task 2 — list rows where production is NA:

```
#| label: missing-list
na_mask <- is.na(meat$production)
meat[na_mask, c("country", "animal", "year")]
```

### Verifier cross-check:

```
#| label: missing-verify
nrow(meat[!is.na(meat$production), ]) # Should equal nrow(meat) minus count of NAs
```

### Task 3 — total production without and with `na.rm`:

```
#| label: missing-sum
sum(meat$production) # Returns NA
sum(meat$production, na.rm = TRUE) # Returns a number
```

**Task 4 (written reflection):** Using `na.rm = TRUE` assumes the missing rows contribute zero to the total, or equivalently that those country-animal-year combinations had negligible production. Whether this is justifiable depends on *which* rows are missing — if they are small producers (e.g., Singapore for certain meats), it may be acceptable, but if a major producer like Indonesia has an NA for a key category, the total would be significantly underestimated.

## Section 3.3 — Compare Countries in 2020

### Task 1 — max poultry producer in 2020:

```
#| label: max-poultry
poultry_2020 <- meat[meat$animal == "poultry" & meat$year == 2020, ]
```

```
max_prod <- max(poultry_2020$production, na.rm = TRUE)
poultry_2020$country[poultry_2020$production == max_prod]
```

**Verifier cross-check using `min_rank()`:**

```
#| label: max-poultry-verify
ranks <- min_rank(desc(poultry_2020$production))
poultry_2020$country[ranks == 1]
```

**Task 2 — second-highest:**

```
#| label: second-poultry
poultry_2020$country[ranks == 2]
```

**Task 3 — how many countries produced more poultry than pig in 2020:**

```
#| label: poultry-vs-pig
poultry_2020 <- meat[meat$animal == "poultry" & meat$year == 2020, ]
pig_2020      <- meat[meat$animal == "pig"      & meat$year == 2020, ]

# Confirm same country order before comparing
identical(poultry_2020$country, pig_2020$country)

# Count countries where poultry > pig
sum(poultry_2020$production > pig_2020$production, na.rm = TRUE)
```

⚠ Only proceed with the comparison after confirming `identical()` returns TRUE.

## Section 3.4 — Construct Factors

**Task 1 — convert `animal` to factor:**

```
#| label: factor-create
meat$animal_factor <- factor(meat$animal)
is.factor(meat$animal_factor)
```

**Task 2 — check levels and alphabetical order:**

```
#| label: factor-levels
levels(meat$animal_factor)

# Check if levels are in alphabetical order
alpha_levels <- sort(levels(meat$animal_factor))
identical(levels(meat$animal_factor), alpha_levels)
# OR using min_rank:
identical(min_rank(levels(meat$animal_factor)), 1:length(levels(meat$animal_factor)))
```

**Task 3 (written answer):** `meat$animal_factor` should **not** be an ordered factor. The four animal categories ("beef and buffalo", "mutton and goat", "pig", "poultry") are **nominal** — there is no natural ordering between them (e.g., beef is not "greater than" poultry in any meaningful sense). An ordered factor implies a ranked hierarchy, which does not apply here.

**Task 4 — collapse into red/white using `fct_collapse()`:**

```
#| label: factor-color
meat$color <- fct_collapse(
  meat$animal_factor,
  red   = c("beef and buffalo", "mutton and goat", "pig"),
  white = "poultry"
)
levels(meat$color) # Should show: "red" "white"
```

## Section 3.5 — Add a Country-Code Column

Use `recode_values()` (from the `dplyr` package, part of `tidyverse`). The `~` formula syntax maps old values to new values:

```
#| label: iso3-recode
meat$iso3 <- recode_values(
  meat$country,
  "Brunei Darussalam" ~ "BRN",
  "Cambodia"          ~ "KHM",
  "Indonesia"         ~ "IDN",
  "Lao PDR"           ~ "LAO",
  "Malaysia"          ~ "MYS",
  "Myanmar"           ~ "MMR",
  "Philippines"       ~ "PHL",
  "Singapore"        ~ "SGP",
  "Thailand"          ~ "THA",
  "Viet Nam"         ~ "VNM"
)
```

**Confirm no NAs and 10 unique pairs:**

```
#| label: iso3-verify
any(is.na(meat$iso3)) # Should be FALSE
length(unique(paste(meat$country, meat$iso3))) # Should be 10
```

## Key Reminders for Submission

- Every code chunk needs a **unique** `#| label:`
- Use `|>` (pipe) when chaining, `::` when qualifying namespaces (e.g., `forcats::fct_collapse()`)
- No `case_match()` — use `recode_values()` or `replace_values()` as shown above

- Use the **Air formatter** to clean up code formatting before submitting
- Include `ai_log.md` in the ZIP archive with every prompt and response verbatim
- Test by unzipping your archive in a fresh folder and re-rendering the QMD before uploading

\*\*

## <q>Section 3.4 — Construct FactorsTask 1 — convert animal to factor:r#| label: factor-create

```
meatanimal_factor <- factor(meatanimal)
is.factor(meatanimal_factor)
```

### Check if levels are in alphabetical order

```
alpha_levels <- sort(levels(meatanimal_factor))
identical(levels(meatanimal_factor),
alpha_levels)
```

### OR using min\_rank:

```
identical(min_rank(levels(meatanimal_factor)), 1 : length(levels(meat
animal_factor)))Task 3 (written answer): meatcolor <- fct_collapse(
meat
animal_factor, red = c("beefandbuffalo", "muttonandgoat", "pig"), white = "poultry")
# Should show: "red" "white"Section 3.5 — Add a Country-Code ColumnUse
recode_values() (from the dplyr package, part of tidyverse). The ~ formula syntax maps old
values to new values:r#| label: iso3-recode
meatiso3 <- recode_values(meatcountry,
"Brunei Darussalam" ~ "BRN",
"Cambodia" ~ "KHM",
"Indonesia" ~ "IDN",
"Lao PDR" ~ "LAO",
"Malaysia" ~ "MYS",
"Myanmar" ~ "MMR",
"Philippines" ~ "PHL",
"Singapore" ~ "SGP",
"Thailand" ~ "THA",
"Viet Nam" ~ "VNM")
# Confirm no NAs and 10 unique pairs:r#| label: iso3-verify
any(is.na(meatcountry, meat$iso3)) # Should be 0</q> Based on the instructions of the
labsheet for section 3.4 as seen below:
```

### 3.4 Construct Factors

1. Convert `meatanimal` to a factor and assign the result to a new column `meatanimal_factor`.

Confirm with `is.factor()`.

2. Report the result of `levels(meat$animal_factor)`. Use `min_rank()` and `identical()` to check if the levels are ordered alphabetically.
3. Should `meat$animal_factor` be an ordered factor? Why or why not?
4. Using `fct_collapse()` on `meat$animal_factor`, *create a new factor* `meat$color` in which "beef and buffalo", "mutton and goat", and "pig" are merged into a single level called "red", while "poultry" becomes "white". Use `levels(meat$color)` to confirm that the new factor has two levels.

It sounds to me like a new column should be made however the code that you gave me :

```
#| label: factor-create
meat$animal_factor <- factor(meat$animal)
is.factor(meat$animal_factor)
```

You're right to question this — and your reading of the lab instructions is correct. Let me clarify exactly what's happening and why the code is correct.

## The Code Is Creating a New Column

```
meat$animal_factor <- factor(meat$animal)
```

This line **does** create a new column. Here's the breakdown:

- `meat$animal` — reads the **existing** `animal` column (a character vector)
- `factor(...)` — converts it to a **factor**
- `meat$animal_factor <-` — assigns the result to a **brand new column** called `animal_factor`

After this line, `meat` will have **5 columns**: `country`, `animal`, `year`, `production`, and the new `animal_factor`. The original `animal` column is **untouched** — this is important because `fct_collapse()` in Task 4 also operates on `meat$animal_factor`, not `meat$animal`.<sup>[12]</sup> <sup>[13]</sup>

## Why Keep Both Columns?

The lab asks you to keep `meat$animal` (character) and create `meat$animal_factor` (factor) as a separate column so that:

1. You can verify the conversion was correct by comparing them side-by-side
2. `fct_collapse()` in Task 4 then builds `meat$color` **from** `meat$animal_factor` <sup>[14]</sup>

## The Full Flow for Section 3.4

```
#| label: factor-create
# Task 1: New column animal_factor (meat now has 5 columns)
meat$animal_factor <- factor(meat$animal)
is.factor(meat$animal_factor) # Should return TRUE
```



```

#| label: factor-levels
# Task 2: Report levels and check alphabetical order
levels(meat$animal_factor)
# factor() sorts levels alphabetically by default [web:13]
# So levels should be: "beef and buffalo" "mutton and goat" "pig" "poultry"

alpha_levels <- sort(levels(meat$animal_factor))
identical(levels(meat$animal_factor), alpha_levels) # Should be TRUE

```

```

#| label: factor-color
# Task 4: New column color (meat now has 6 columns)
meat$color <- fct_collapse(
  meat$animal_factor,
  red   = c("beef and buffalo", "mutton and goat", "pig"),
  white = "poultry"
)
levels(meat$color) # Should show: "red" "white"

```

## Column Growth Summary

| After step                    | Columns in meat                   |
|-------------------------------|-----------------------------------|
| After <code>read_csv()</code> | country, animal, year, production |
| After Task 1                  | + animal_factor                   |
| After Task 4                  | + color                           |
| After Section 3.5             | + iso3                            |

So `meat$animal_factor <- factor(meat$animal)` is the correct syntax for Task 1 — it creates a new column while leaving `meat$animal` intact.<sup>[13]</sup>

✱✱

**Could you just help verify for me that only the permitted functions and formats are being used as I had specified initially? Could you also verify for me that the code is to be working as expected.**

Good news — almost everything is compliant. The audit caught 2 violations that need fixing. Here is the full corrected code for those specific chunks, plus the clean final versions of all sections.

## ⚠ Two Violations Found & Fixed

### ✖ Violation 1: `sort()` in `factor-levels`

`sort()` is not in the permitted toolbox. The fix is to build the alphabetical reference vector manually using `c()` and compare with `identical()`.

Before (incorrect):

```
alpha_levels <- sort(levels(meat$animal_factor))
identical(levels(meat$animal_factor), alpha_levels)
```

After (corrected):

```
#| label: factor-levels
levels(meat$animal_factor)
# factor() assigns levels alphabetically by default.
# Manually specify expected alphabetical order using c():
alpha_levels <- c("beef and buffalo", "mutton and goat", "pig", "poultry")
identical(levels(meat$animal_factor), alpha_levels)

# Second check using min_rank():
identical(
  min_rank(levels(meat$animal_factor)),
  1:length(levels(meat$animal_factor))
)
```

### ✖ Violation 2: `paste()` in `iso3-verify`

`paste()` is not in the permitted toolbox. The fix uses only `unique()` and `length()` applied directly to each column, plus a `sum() + is.na()` check for NAs.

Before (incorrect):

```
length(unique(paste(meat$country, meat$iso3))) # paste() not permitted
```

After (corrected):

```
#| label: iso3-verify
# Check no NAs in iso3
any(is.na(meat$iso3))          # Should be FALSE

# Confirm 10 unique country names and 10 unique iso3 codes
length(unique(meat$country))   # Should be 10
length(unique(meat$iso3))      # Should be 10
```

## ✓ Full Audit Summary

| Chunk Label        | Functions Used                                   | Status  |
|--------------------|--------------------------------------------------|---------|
| setup-meat         | library(), read_csv()                            | ✓ OK    |
| inspect-classes    | class()                                          | ✓ OK    |
| inspect-shape      | nrow(), ncol()                                   | ✓ OK    |
| inspect-unique     | length(), unique()                               | ✓ OK    |
| inspect-glimpse    | glimpse()                                        | ✓ OK    |
| inspect-headtail   | head(), tail()                                   | ✓ OK    |
| missing-confirm    | any(), is.na()                                   | ✓ OK    |
| missing-list       | is.na(), [, \$                                   | ✓ OK    |
| missing-verify     | is.na(), nrow()                                  | ✓ OK    |
| missing-sum        | sum()                                            | ✓ OK    |
| max-poultry        | max(), ==, &, [, \$                              | ✓ OK    |
| max-poultry-verify | min_rank(), desc()                               | ✓ OK    |
| second-poultry     | min_rank(), []                                   | ✓ OK    |
| poultry-vs-pig     | identical(), sum(), >                            | ✓ OK    |
| factor-create      | factor(), is.factor()                            | ✓ OK    |
| factor-levels      | levels(), c(), identical(), min_rank(), length() | ✓ Fixed |
| factor-color       | fct_collapse(), levels()                         | ✓ OK    |
| iso3-recode        | recode_values(), ~                               | ✓ OK    |
| iso3-verify        | any(), is.na(), length(), unique()               | ✓ Fixed |

✱

1. [https://dplyr.tidyverse.org/reference/case\\_match.html](https://dplyr.tidyverse.org/reference/case_match.html)
2. <https://dplyr.tidyverse.org/reference/recode-and-replace-values.html>
3. <https://stackoverflow.com/questions/78236470/recode-values-using-case-match-with-a-char-array>
4. <https://rdr.io/github/hadley/dplyr/man/recode.html>
5. [https://cghlewis.com/blog/dplyr\\_update/](https://cghlewis.com/blog/dplyr_update/)
6. [https://www.ericeikrem.com/r-programming-goodbye-recode-hello-case\\_match/](https://www.ericeikrem.com/r-programming-goodbye-recode-hello-case_match/)
7. <https://search.r-project.org/CRAN/refmans/poorman/html/recode.html>
8. [https://rdr.io/cran/dplyr/man/case\\_match.html](https://rdr.io/cran/dplyr/man/case_match.html)
9. [https://ericeikrem.com/r-programming-goodbye-recode-hello-case\\_match/](https://ericeikrem.com/r-programming-goodbye-recode-hello-case_match/)
10. <https://cran.rstudio.com/web/packages/dplyr/vignettes/recoding-replacing.html>

11. [Week2\\_GrpLab\\_Guide.pdf](#)
12. [https://www.youtube.com/watch?v=xkRBfy8\\_2MU](https://www.youtube.com/watch?v=xkRBfy8_2MU)
13. <https://www.geeksforgeeks.org/r-language/r-factors/>
14. <https://r4ds.had.co.nz/factors.html>
15. <https://github.com/datacamp/courses-introduction-to-r/blob/master/chapter4.Rmd>
16. <https://r-coder.com/factor-r/>
17. <https://stats.oarc.ucla.edu/r/modules/factor-variables/>
18. <http://monashbioinformaticsplatform.github.io/2015-09-28-rbioinformatics-intro-r/01-sup-factors.html>
19. [https://www.stat.ubc.ca/~jenny/STAT545A/block08\\_bossYourFactors.html](https://www.stat.ubc.ca/~jenny/STAT545A/block08_bossYourFactors.html)
20. <https://www.stat.berkeley.edu/~s133/factors.html>
21. <https://r4ds.hadley.nz/factors.html>
22. <https://stackoverflow.com/questions/1125446/how-do-you-verify-that-users-requirements-are-addressed-in-the-code-youre-work>
23. <https://learn.microsoft.com/en-gb/answers/questions/5528584/not-able-to-run-my-first-azure-function-a-pp-check>
24. <https://discuss.google.dev/t/the-code-change-produced-by-gemini-could-not-be-fully-parsed-cant-automatically-accept-changes/190036>
25. <https://community.openai.com/t/retrievals-tool-not-permitted/628664>
26. <https://github.com/google-gemini/gemini-cli/issues/15428>
27. <https://ask.gov.sg/mindef/questions/cmh4fdaqf00ttuk1bepdiky2v>
28. <https://support.google.com/googleplay/android-developer/thread/354985646/issue-found-unable-to-verify-core-functionality-of-app?hl=en>
29. <https://www.pass4success.com/salesforce/javascript-developer-i/dumps>
30. <https://www.marks4sure.com/acd201-appian-senior-developer-questions.html>